

ASE 420 Tools Installation - VSCode

VSCode is Your **Weapon of Choice**

In the age of AI, many IDE tools have emerged, but VS Code is the first tool that software engineers should master.

Install VS Code

- [Download VS Code | code.visualstudio.com](https://code.visualstudio.com)
- After installing, open VS Code once to confirm it launches correctly before continuing.

| VS Code Extensions

- We use Visual Studio Code as the primary IDE.
 - Install the Python extension.
 - Install any other extension needed for your project.

Why VS Code for ASE 420?

- **Python support:** first-class interpreter selection, linting, and IntelliSense.
- **Jupyter:** run notebooks without leaving the editor.
- **Debugger:** set breakpoints and step through code visually.
- **Git integration:** clone, commit, and review changes in the same window.

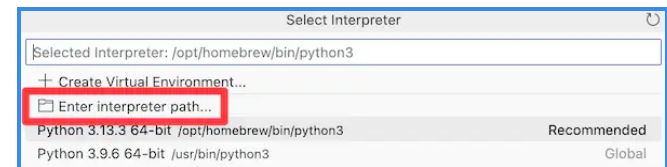
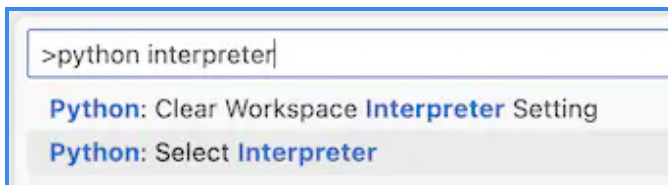
Run the Tetris programs

- Clone the ASE420 course repository and open the Tetris example.

```
git clone https://github.com/nkuase/ase420
cd ase420/course_projects/code
pip install pygame
python tetris_ver1.py
```

Step 1: Select the Python Interpreter

- Install the Python extension in VS Code.
- Open the Tetris program in VS Code.
 - Select the interpreter from the project's virtual environment (status bar, or the **Python: Select Interpreter** command). See [VS Code docs](#).



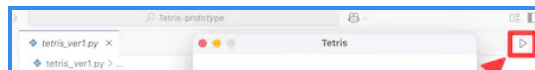
Step 2: Run the App

- Click the arrow to run the app, or run it from the terminal.
- **Recommended:** install `pygame` with UV.

```
uv add pygame  
uv run tetris_ver1.py
```

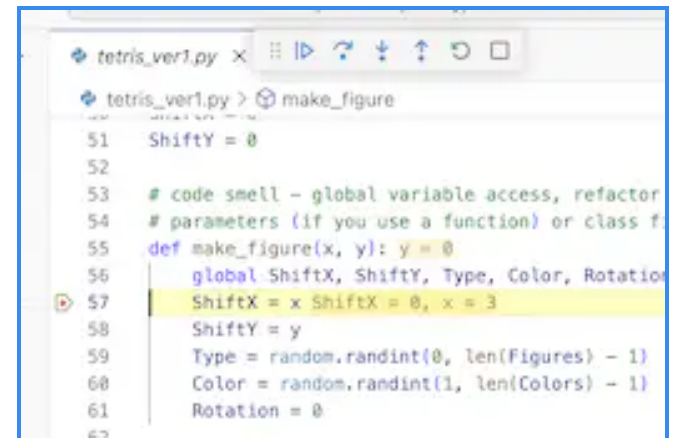
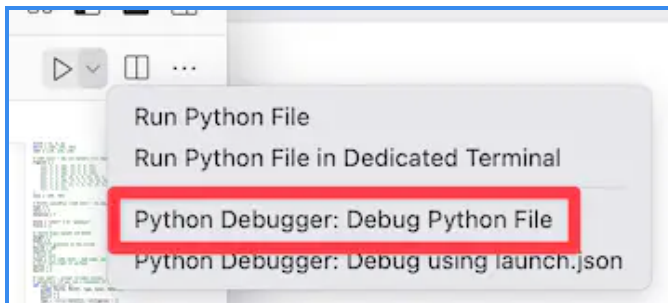
- **Alternative:** install `pygame` with `pip` in your `venv`.

```
pip install pygame  
python tetris_ver1.py
```



Step 3: Debug with Breakpoints

- Use VS Code: choose the Python debugger and set breakpoints.



Use Jupyter

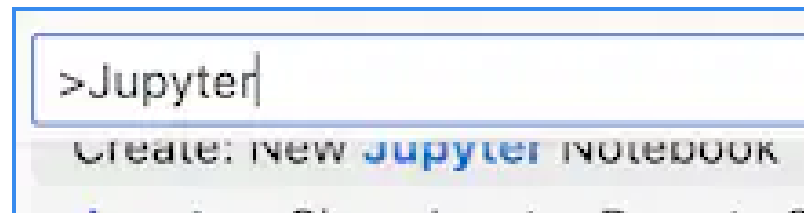
| What Problems Does Jupyter Solve?

1. Combine code, explanation, and results
2. Develop and test code interactively

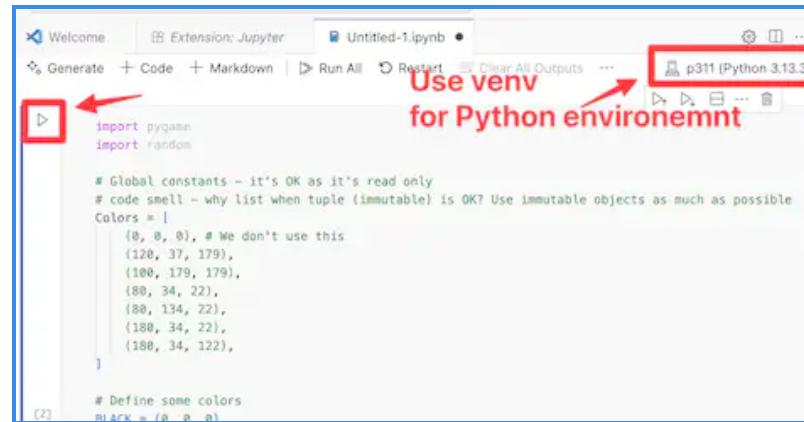
What Benefits Does Jupyter Give?

- Combines code, output, and explanation in one notebook, so notes and results stay next to the code that produced them.
- Runs and re-runs small pieces of code (cells) independently, without restarting the whole program.
- Well suited for exploring and visualizing intermediate results, such as testing Tetris logic step by step.

- Install Jupyter extension & set Python environment as your venv Python.



- Use Jupyter to learn Python and analyze the Tetris program.



Use Profile

| What Problems Does Profile Solve?

- Switching projects manually reconfigures extensions and settings, which is slow and error-prone.
- A **Profile** saves and switches named collections of extensions, settings, and layouts for each context (e.g., Python vs. web).

What Benefits Does Profile Give?

- **Isolation:** Course-specific settings don't leak into your personal or other project setups.
- **Fast switching:** Move between contexts (e.g., "ASE420", "Web Dev") with one click instead of reconfiguring VS Code.
- **Shareable:** Export a profile so teammates or students start with the same setup.

Create an ASE420 Profile

- Create a dedicated VS Code profile for this course containing the Python extension, debugger, and Jupyter extension.
- This keeps your course environment isolated and easy to reset if something goes wrong.

Use Workspace

| What Problems Does Workspace Solve?

- Opening project folders separately breaks search, IntelliSense, and settings continuity.
- A **multi-root workspace** combines multiple related folders into a single VS Code session.
- Use it when your project spans multiple folders; a single-folder project usually doesn't need one.

What Benefits Does Workspace Give?

- **Unified search:** Search and navigate across all folders at once.
- **Consistent settings:** Share extensions, settings, and debug configurations across the whole workspace.
- **Fewer window switches:** Work on related repositories side by side in one window.

Example: Multi-root Workspace

```
{  
  "folders": [  
    {  
      "path": "./course_projects/src"  
    },  
    {  
      "path": "./course_projects/code"  
    }  
  ]  
}
```

Use Copilot Student

NKU students verified through the **GitHub Student Developer Pack** get free access to **GitHub Copilot Student**.

What Problems Does Copilot Solve?

- Writing boilerplate and repetitive code by hand slows development.
- Looking up syntax or API details in documentation interrupts your flow.
- **GitHub Copilot** solves this with AI-based code completion built directly into the editor.

What Benefits Does Copilot Give?

- **Faster typing:** Auto-completes repetitive code such as loops, boilerplate, and common patterns.
- **Inline help:** Suggests function usage and syntax without leaving the editor.
- **Free for NKU students:** Verify through the GitHub Student Developer Pack to get GitHub Copilot Student at no cost. (Plan details and availability can change — check the [current GitHub Education page](#).)

Focus on the problem, not the tool

- Copilot suggestions are not always correct; always read and understand code before accepting it.
- You are still responsible for the code you submit, including anything Copilot generated.
- Use Copilot to move faster, not to skip understanding the problem.